30_API_SPEC.md

UNPLND API Specification

Version 1.0

Priority: CRITICAL

Status: MVP

Purpose

This document defines all backend APIs used by UNPLND.

The API layer exists to protect the client from complexity.

The app should never directly coordinate AI, maps, weather and validation by itself.

The backend orchestrates.

The client displays.

API Philosophy

Simple request.

Smart backend.

Beautiful response.

The frontend should never need to understand complex route generation logic.

Base URL Structure

Production

/api/v1

Staging

/api/staging/v1

Development

/api/dev/v1

Always version APIs.

Never break older clients without migration.

Authentication

All protected endpoints require Firebase Authentication token.

Header

Authorization: Bearer {token}

Guest users may access limited planner preview endpoints.

Core Services

Planner Service

AI Service

Maps Service

Weather Service

Album Service

Notification Service

User Service

Premium Service

Analytics Service

POST /plans/generate

Purpose

Generate a complete evening plan.

Request

{ "location": { "lat": 0, "lng": 0, "city": "Gießen" }, "mood": "cozy", "budget": 50, "groupSize": 2, "transport": "walking", "availableTime": 4, "preferences": ["indoor", "quiet"] }

Response

{ "planId": "string", "summary": "A cozy evening with dinner, drinks and a quiet viewpoint.", "estimatedCost": 46, "estimatedDuration": "4h", "experienceScore": 87, "route": [] }

POST /plans/{planId}/regenerate

Purpose

Regenerate entire route or part of route.

Request

{ "mode": "entire_route", "keepCompletedStops": true }

Modes

entire_route

remaining_route

single_stop

category

POST /plans/{planId}/replace-stop

Purpose

Replace a single stop with better alternatives.

Request

{ "stopId": "string", "reason": "too_expensive" }

Response

{ "newStop": {}, "updatedRoute": {}, "experienceScore": 89 }

POST /plans/{planId}/complete-stop

Purpose

Mark route stop as completed.

Request

{ "stopId": "string", "completedAt": "server_timestamp" }

POST /plans/{planId}/complete

Purpose

Finish evening and trigger album creation suggestion.

Response

{ "completed": true, "suggestAlbum": true }

GET /plans/{planId}

Returns one plan.

Must verify ownership.

GET /plans

Returns user's plans.

Supports

pagination

status

date range

POST /albums/create

Purpose

Create memory album from completed plan.

Request

{ "planId": "string", "photoIds": [], "title": "Summer Night in Frankfurt" }

Response

{ "albumId": "string", "coverUrl": "string", "status": "created" }

PATCH /albums/{albumId}

Purpose

Edit album metadata.

Fields

title

mood

notes

favorite

coverPhoto

GET /albums/{albumId}

Returns album.

Owner only unless shared link exists.

POST /albums/{albumId}/share

Creates share link.

Privacy

private by default.

DELETE /albums/{albumId}

Soft delete first.

Hard delete later.

GET /discover

Returns curated inspiration.

Query

city

season

mood

budget

distance

GET /search

Global search.

Searches

places

collections

albums

favorites

Supports natural language later.

POST /favorites/add

Add place, route, collection or album.

POST /favorites/remove

Remove favorite.

GET /weather

Purpose

Get weather for planning context.

Cached for 15 minutes.

GET /places/nearby

Purpose

Search verified nearby places.

Backend wraps Google Places.

Client should not call Places directly when avoidable.

POST /notifications/register-device

Registers FCM token.

POST /feedback

User feedback.

Bug reports.

Feature suggestions.

Error Format

All errors return

{ "error": { "code": "string", "message": "friendly message", "retryable": true } }

Never expose stack traces.

Error Codes

AUTH_REQUIRED

PERMISSION_DENIED

VALIDATION_FAILED

PLAN_GENERATION_FAILED

NO_VALID_ROUTE

RATE_LIMITED

NETWORK_ERROR

SERVER_ERROR

Rate Limiting

Plan generation

Free users

limited

Premium users

higher limit

Guest users

strict limit

Rate limits must prevent abuse.

Validation

Validate every input server-side.

Never trust client.

Budget must be positive.

Coordinates must be valid.

Mood must be allowed value.

Transport must be supported.

Security

Never expose API keys.

Never expose raw AI prompts.

Never expose provider credentials.

All provider calls happen server-side.

Response Time Targets

Standard API

<300ms

Plan generation

<8 seconds

Album creation

<5 seconds

Search

<500ms

Logging

Log

Request ID

User ID

Latency

Error Code

Never log

Passwords

Private notes

Photo content

Precise sensitive data

API Versioning

All endpoints under /v1.

Breaking changes require /v2.

Maintain backward compatibility.

Claude Instructions

Frontend never talks directly to AI providers.

Frontend never exposes secrets.

Backend owns orchestration.

APIs must be predictable, documented and testable.

Never create undocumented endpoints.

END OF 30_API_SPEC.md